ML Sentry: An Agentic, Model-Agnostic Governance Layer for Monitoring Deployed Machine Learning Models

Paper input pack: Sections III-VI (Dataset, Methodology, Experimental Setup, Results), written in the paper's format. Abstract, Introduction and Literature Review are in the main manuscript. All numbers come from the project's test runs of 26-28 October 2026.

III. Dataset

ML Sentry is model-agnostic: it reads only the evaluation metrics that a deployed model logs to MLflow, never the model or its data. A dataset is therefore needed only to build a realistic demonstration model whose metrics ML Sentry can watch. Since no real-time banking data was available to the project, the public California Housing dataset was used, and production faults were injected into it in a controlled way (Section III-E).

A. Source and Description

The California Housing dataset is derived from the 1990 U.S. Census and is distributed with scikit-learn [cite: Pace & Barry, 1997; Pedregosa et al., 2011]. Each record describes one census block group.

- **Record count:** 20,640 block groups, with no missing values.
- **Features (8, numeric):** median income (MedInc, in $10,000s), house age, average rooms, average bedrooms, population, average occupancy, latitude and longitude.
- **Target:** median house value (MedHouseVal) in units of $100,000, capped at 5.0; mean 2.07.
- **Task type:** regression. This matches the scope of the team's governance layer, which currently supports regression models.

B. Group Attribute for the Bias Check

A fairness check needs a group attribute. Since the dataset contains no demographic fields, a geographic proxy was derived: a block group is labelled *coastal* when its longitude is below -121.0 and *inland* otherwise. This gives 7,143 coastal and 13,497 inland records. The attribute is used only to group the evaluation error; it is not given to the model as an input feature.

C. Data Partitions

The data was split once with a fixed seed (42) into three disjoint parts, each with its own role (Table I).

TABLE I

DATA PARTITIONS AND THEIR ROLES

Partition	Rows	Coastal / inland	Role
Train	12,384 (60%)	4,340 / 8,044	Fit the baseline model
Reference	4,128 (20%)	1,369 / 2,759	Healthy reference for metrics and drift
Live	4,128 (20%)	1,434 / 2,694	Replayed as production traffic

D. Live Stream Simulation

To imitate a model in production, the live partition is replayed in eight consecutive batches of 500 rows (the remaining 128 rows are not used). Each batch is scored by the model and its metrics are logged to MLflow as one run, so each batch is one evaluation result that ML Sentry must check.

E. Fault Injection

Three kinds of production fault were injected into copies of the live batches (Table II). In every scenario, batches 1-3 are left healthy and the fault is applied from batch 4 onward, so the moment of failure is known exactly. The original data is never modified.

TABLE II

INJECTED FAULTS

Scenario	What is changed	Real-world meaning
Feature drift	MedInc + 2.0 ($20,000) on every row	New population, unlike the training data
Performance decay	Target x 1.3 on every row	Prices rose 30%; inputs unchanged (concept drift)
Group bias	Target x 1.4 for coastal rows only	Model becomes much worse for one group

IV. Methodology

ML Sentry is a governance layer that runs beside a model already in production. It follows four sequential phases: (1) the user's model logs evaluation metrics to MLflow, (2) ML Sentry ingests new results and stores their history, (3) every metric is compared with its limit, and (4) an LLM agent judges the severity of any breach and an alert is sent. The model is never uploaded, moved or loaded by ML Sentry.

A. System Overview and Architecture

Fig. 1 shows the four phases. Phase 1 belongs to the user's side; phases 2-4 are ML Sentry, packaged as a Docker container configured by a single `.env` file. When the user retrains the model, the new results flow through the same path, so recovery is checked automatically.

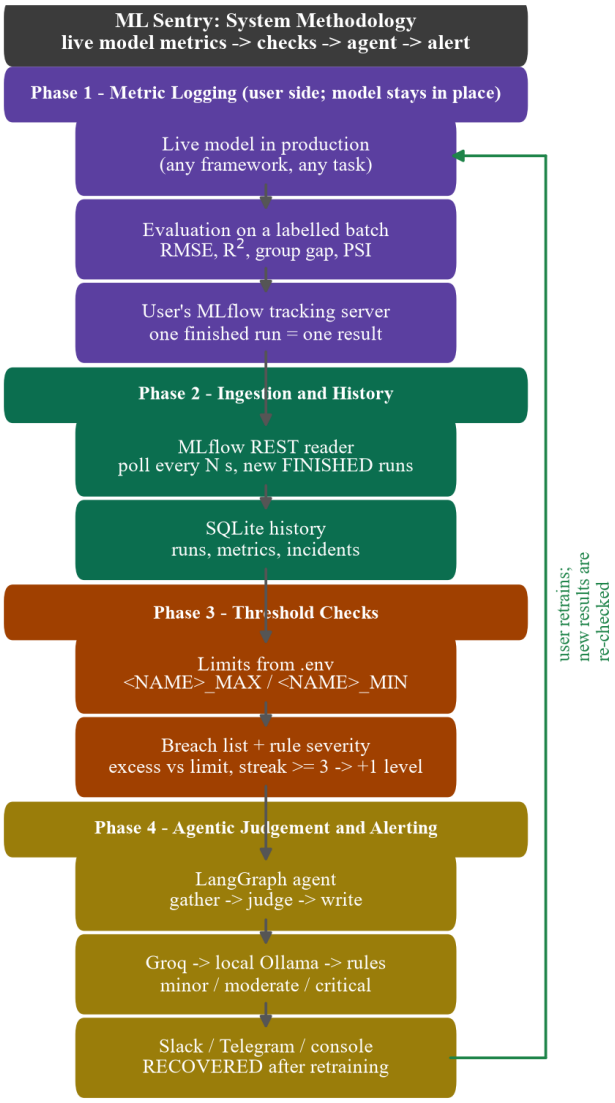


Fig. 1. System methodology: the four-phase pipeline from metric logging to alerting, with the retraining feedback loop.

B. Phase 1: Metric Logging

For every evaluation batch the model's owner logs metrics to MLflow. In the demonstration, a simulator logs 16 metrics per batch, of which ML Sentry watches four. The prediction error and goodness of fit are

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{1}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{2}$$

For regression there is no approve/reject decision, so group parity measures for classifiers do not apply. Bias is therefore measured as the gap between the group errors:

$$\mathrm{gap} = \max_g \mathrm{RMSE}_g - \min_g \mathrm{RMSE}_g \tag{3}$$

Data drift of each input feature is measured with the Population Stability Index between the reference data (expected bin shares e_k) and the batch (actual shares a_k), computed with Evidently [cite: Evidently AI]; the largest value over the eight features is logged as *max_psi*:

$$\mathrm{PSI} = \sum_k (a_k - e_k) \ln \frac{a_k}{e_k} \tag{4}$$

C. Phase 2: Ingestion and History

- **Reading:** every N seconds ML Sentry calls MLflow's REST API (runs/search) for FINISHED runs in the configured experiment. It needs neither the model nor the MLflow Python package.
- **Deduplication:** run IDs already stored are skipped; a one-hour look-back catches runs that started earlier but finished late.
- **History:** each run, its metrics and every incident are stored in SQLite (tables runs, metrics, incidents).
- **First start:** runs that already exist when ML Sentry first starts are stored as history without alerts, so the user is not flooded with alerts about the past.

D. Phase 3: Threshold Checks

Each watched metric has a limit in `.env`: `<NAME>_MAX` for lower-is-better metrics (errors, drift) and `<NAME>_MIN` for higher-is-better metrics (R^2, accuracy). A breach is any value beyond its limit L; its relative excess is

$$\varepsilon = \frac{|v - L|}{|L|} \tag{5}$$

A transparent rule gives a first severity from the worst breach: *minor* if epsilon < 0.25, *moderate* if 0.25 <= epsilon < 1.0, and *critical* otherwise. A problem that persists for three or more runs in a row is raised by one level. This rule is the hint given to the agent and the final fallback when no LLM answers.

E. Phase 4: Agentic Judgement and Alerting

The agent is a LangGraph state graph with three nodes (Fig. 2) [cite: LangGraph]:

- **gather** collects the last ten values of each breached metric and the bad-run streak;
- **judge** asks an LLM for a severity, a reason and an action, as JSON at temperature 0. The answer is validated (the severity must be minor, moderate or critical). The LLMs are tried in order: Groq (openai/gpt-oss-120b), then a local Ollama model (llama3.1:8b), then the rule of Section IV-D;
- **write** formats the alert: severity, each metric against its limit, why, and what to do.

The LLM sees only metric names and numbers, never the model or the data, which keeps the agent model-agnostic. Alerts are always printed to the console and stored in SQLite, and are sent to Slack (incoming webhook) and/or Telegram; a failed channel never stops monitoring.

F. Recovery and Re-checking

When a run passes all limits after one or more breaching runs, ML Sentry sends a RECOVERED message. After the user retrains, the new model's results are checked in exactly the same way: a lasting problem keeps alerting, a fixed one produces a recovery message.

V. Experimental Setup

A. Hardware and Software

TABLE III
HARDWARE AND SOFTWARE

Component	Detail
Machine	Windows 11 laptop; Intel Core i5-12450HX (8 cores, 12 threads); 16 GB RAM; NVIDIA GeForce RTX 4050 Laptop GPU
Language	Python 3.14.3 (virtual environment); Python 3.12 inside the Docker image
Data and model	pandas 3.0.6, scikit-learn 1.9.1, scipy 1.18.1
Drift measure	Evidently 0.7.23 (PSI)
Metric tracking	MLflow 3.17.2 (local tracking server, SQLite backend)
Agent	LangGraph 1.2.12; Groq API (openai/gpt-oss-120b); Ollama (llama3.1:8b, run locally, split across GPU and CPU)
History	SQLite
Alerts	Console; Slack incoming webhook; Telegram (supported, not tested)
Packaging	Docker and docker-compose; image 294 MB (python:3.12-slim base)
Tests	pytest: 9 automated tests

B. Baseline Model

A HistGradientBoostingRegressor from scikit-learn was trained on the train partition with its default settings (learning rate 0.1, up to 100 boosting iterations, 31 leaves per tree) and seed 42. Because the train partition has more than 10,000 rows, scikit-learn's automatic early stopping was active: 10% of the train rows were held out internally for validation, and training still ran all 100 iterations. Its inputs are the eight numeric features; the region label is excluded. The model and its feature list are saved together, and its healthy metrics are measured on the reference partition (Section VI-A).

C. Monitoring Configuration

The limits were set with a safety margin outside the healthy range, which over eight healthy live batches was (RMSE 0.422-0.503, R^2 0.822-0.867, gap 0.007-0.142, max PSI 0.032-0.062). A PSI above 0.2 is a common rule of thumb for significant drift. Table IV lists the configuration.

TABLE IV
ML SENTRY CONFIGURATION USED IN THE EXPERIMENTS

Setting	Value
RMSE_MAX	0.60
R2_MIN	0.70
RMSE_GAP_MAX	0.20
MAX_PSI_MAX	0.20
Escalation	+1 severity level after 3 bad runs in a row
Poll interval	10 s in .env.example and the live Groq run; 5 s in the local and Docker runs
LLM order	Groq gpt-oss-120b, Ollama llama3.1:8b, rules
LLM settings	temperature 0; JSON answer; time-outs 30 s (Groq), 180 s (Ollama)
Batches	8 x 500 rows; fault from batch 4

D. Test Protocol

- **Scenarios:** healthy, feature drift, performance decay and group bias, each over eight batches (Table II).
- **Retraining:** for each fault, a new model was trained on the training data with the same fault applied (the "new reality"), the faulty reference data became its new drift reference, and the faulty live batches were replayed through it.
- **Deployments:** local run with Ollama (26 Oct 2026); the same checks inside the Docker container (27 Oct); a live run with Groq and Slack alerts (28 Oct).

E. Evaluation Measures

- **Detection:** whether each faulty batch raised an alert, and false alarms on healthy batches.
- **Detection delay:** faulty batches before the first alert.
- **Alert latency:** seconds from the end of the MLflow run to the alert being sent.
- **Severity agreement:** the LLM's severity compared with the rule-based severity.
- **Recovery:** whether the retrained model's results return within limits.

VI. Results

A. Baseline Performance

Table V gives the baseline model's healthy metrics on the 4,128 reference rows. The error is similar for both groups; the coastal-inland gap of 0.043 is the healthy starting point for the bias check.

TABLE V
BASELINE PERFORMANCE ON THE REFERENCE SET

Group	Rows	RMSE	MAE	R^2
Coastal	1,369	0.505	0.345	0.825
Inland	2,759	0.462	0.308	0.826
Overall	**4,128**	**0.476**	**0.320**	**0.826**

B. Detection of Injected Faults

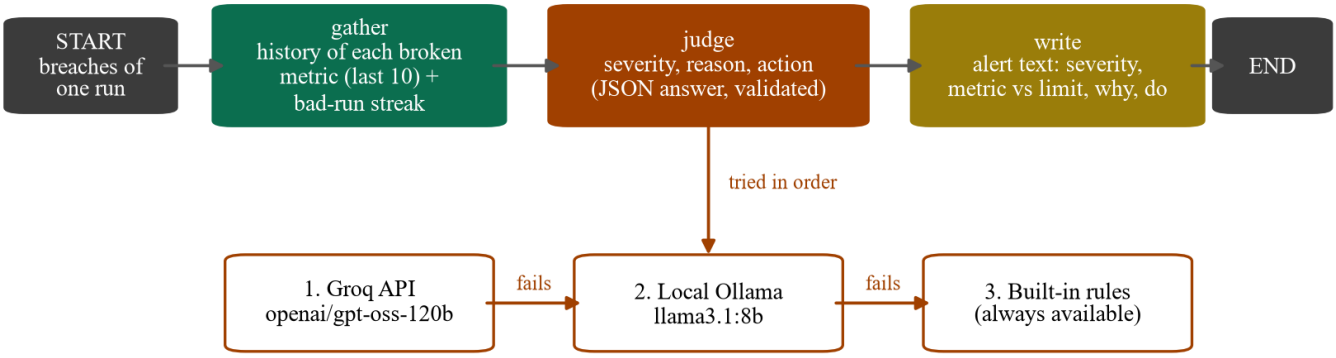


Fig. 2. The LangGraph severity agent: three nodes in sequence. The judge node tries Groq first, then a local Ollama model, then the built-in rules, so an alert is always produced.

Fig. 3 shows all four metrics per batch for the four scenarios. Every fault breaks its own signal at batch 4: drift raises max PSI from at most 0.062 to 2.34-3.00; decay raises RMSE to 0.88-0.99 while PSI stays normal, because the inputs did not change; and bias raises the coastal-inland gap to 0.58-0.77. All three faults also lower R^2 (to 0.33-0.72), showing that one fault usually affects several metrics at once.

Table VI summarises detection. All 15 faulty batches were flagged, each fault on its first faulty batch (delay 0), and no healthy batch raised a false alarm. There are 8 distinct healthy batches: batches 1-3 of the three fault scenarios are the same data as batches 1-3 of the healthy run.

TABLE VI
DETECTION RESULTS PER SCENARIO

Scenario	Metrics breached	Alerts	Delay	Severity per batch 4-8
Healthy	none	0 / 8	-	-
Feature drift	RMSE, R^2, max PSI	5 / 5	0	C C C C C
Perf. decay	RMSE, R^2	5 / 5	0	M M C C C
Group bias	RMSE, gap (+R^2 from 6)	5 / 5	0	C C C C C

(M = moderate, C = critical.) Decay starts as moderate because RMSE is 53% past its limit, and is escalated to critical by the three-runs-in-a-row rule, which is the intended behaviour for a lasting problem.

C. Agent Judgement

Over the three deployments the agent judged 20 breaching batches (Table VII). Each of the 18 severities given by an LLM agreed with the rule-based severity. In the other 2 cases the local Ollama call failed on the first alert of a run and the rules decided: one was a confirmed GPU crash of the Ollama server while loading the model (CUDA error); the cause of the other was not recorded in the log. In both cases the alert was still sent.

TABLE VII
AGENT JUDGEMENTS BY ENGINE

Engine	Judged	Agrees with rules	Alert latency (s)
Groq gpt-oss-120b	5	5 / 5	7.0-14.7 (median 12.9)
Ollama llama3.1:8b	13	13 / 13	20.9-44.3*
Rules (fallback)	2	-	27.6*

* Measured on 9 of the 13 Ollama judgements (bias and Docker runs, poll 5 s; the decay run's times were not kept) and the live Groq run (poll 10 s). A single Groq judgement took about 1.8 s when measured in isolation; most of the latency is the poll wait. Runs are processed one after another, so a slow local LLM delays later batches.

The explanations were correct in substance: Groq attributed the drift alerts to "severe data drift" from the high PSI. The smaller local model once described PSI as a fairness issue, a wrong cause with the right severity, which supports using the larger hosted model first. Fig. 4 shows ML Sentry's console during the live Groq run.

D. Recovery after Retraining

Fig. 5 compares each fault's own signal before and after retraining, and Table VIII gives the outcome. For drift, ML Sentry was running live and sent RECOVERED to Slack. For decay and bias, the retrained models' runs were logged while ML Sentry was not running (on its next start it stored them as history without checking them), so the last column applies ML Sentry's threshold and severity rules (Section IV-D) to the logged metrics.

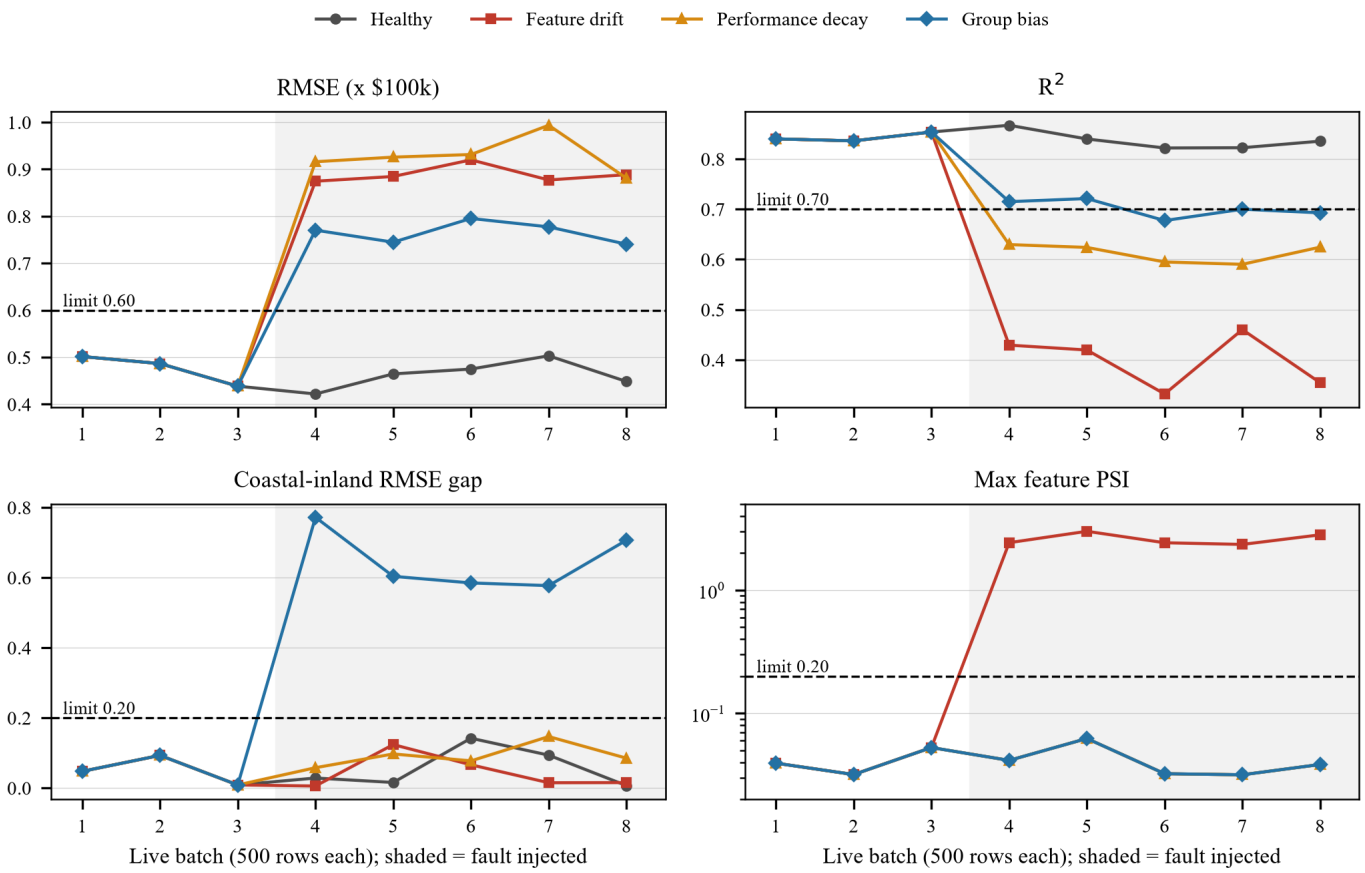


Fig. 3. Metrics of each live batch for the healthy run and the three injected faults (shaded: batches with the fault). Dashed lines are the limits from .env; max PSI uses a log scale.

TABLE VIII
RESULTS AFTER RETRAINING (8 BATCHES EACH)

Scenario	Retrained model	Within limits	ML Sentry outcome
Feature drift	Max PSI 0.032-0.062; RMSE 0.422-0.503	8 / 8	RECOVERED (live, Slack)
Perf. decay	R^2 0.822-0.867; RMSE 0.549-0.654	3 / 8	Alerts again: 4 minor, 1 moderate (rules)
Group bias	Gap 0.010-0.201; RMSE 0.492-0.572	7 / 8	1 minor alert, gap 0.201 (rules)

Drift was fully repaired. For decay, R^2 recovered, but prices are 30% higher, so errors in dollars are 30% larger and RMSE stays near its limit; ML Sentry keeps alerting, which tells the team that the limit or the model must be revisited. This is the re-check after retraining working as intended.

E. Alert Delivery and Deployment

- **Slack:** in the live run all 7 messages (5 CRITICAL, 2 RECOVERED) reached the team channel, with no delivery failure (Fig. 7).
- **MLflow:** the simulator logged 96 finished evaluation runs (97 including one interrupted first attempt) (Fig. 6).
- **Docker:** the 294 MB image reproduced the drift alerts while reading MLflow on the host machine.
- **Tests:** all 9 automated tests passed (Fig. 8).

[Insert screenshot: Slack channel #all-ml-sentry]

Message text delivered in the live run:

```
CRITICAL: run 'groq-slack-drift-batch-4' is out of
bounds
• rmse = 0.8746 (above max 0.6)
• r2 = 0.4293 (below min 0.7)
• max_psi = 2.421 (above max 0.2)
Why: RMSE and R² have degraded far beyond acceptable
limits and max PSI indicates severe data drift ...
Do: Retrain or roll back the model immediately and
monitor after deployment.
(judged by groq, 1 bad run(s) in a row)
```

Fig. 7. Slack alert received in the team channel during the live drift run. [Placeholder: replace with the screenshot from Slack; the text above is the delivered message, shortened (its leading red-circle emoji and the end of the Why line are omitted).]

F. Observations and Limitations

- Limits were chosen from eight healthy batches; a real deployment should set them from a longer history.
- The faults are synthetic; real drift is usually gradual, which the bad-run streak is designed to capture.
- In practice, true labels (e.g. sale prices) arrive late, so error metrics lag while PSI can be checked at once.

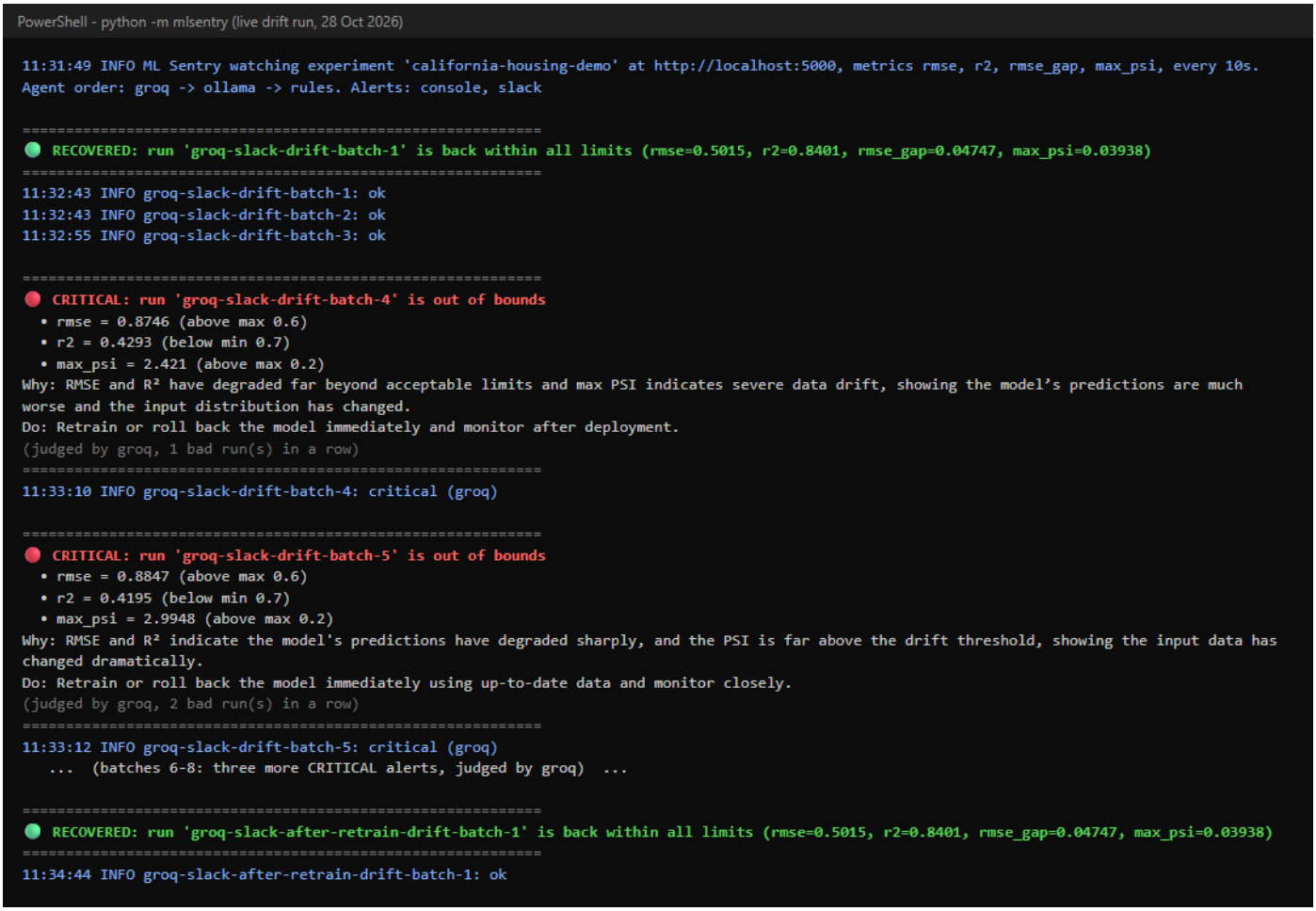


Fig. 4. Console of ML Sentry during the live drift run (28 Oct 2026): healthy batches pass, faulty batches raise CRITICAL alerts judged by Groq with a reason and an action, and RECOVERED is sent after retraining. Captured from the run log; batches 6-8 are shortened.

- Batches are processed sequentially, so a slow local LLM adds latency; with the hosted LLM every alert was sent within 15 s of its run finishing (poll interval 10 s).

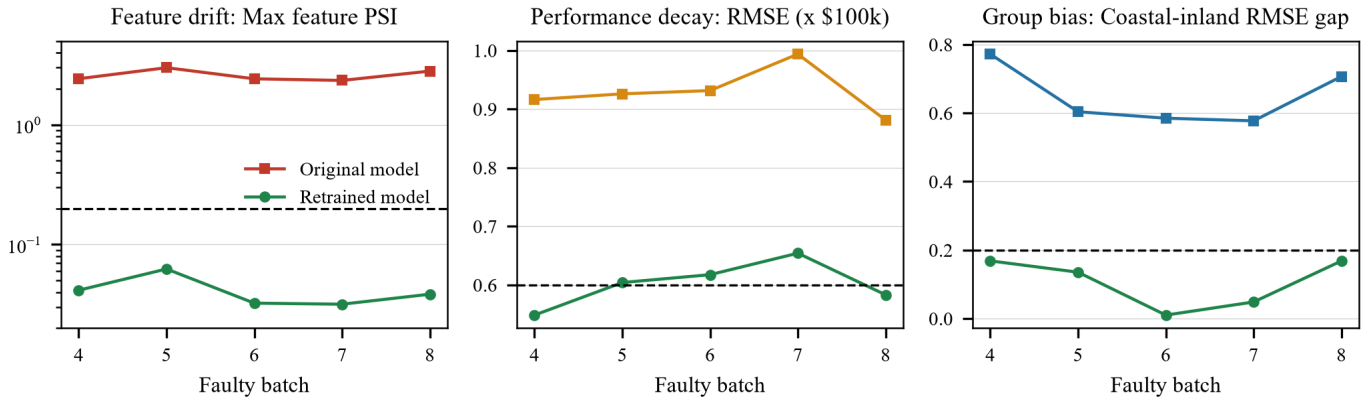
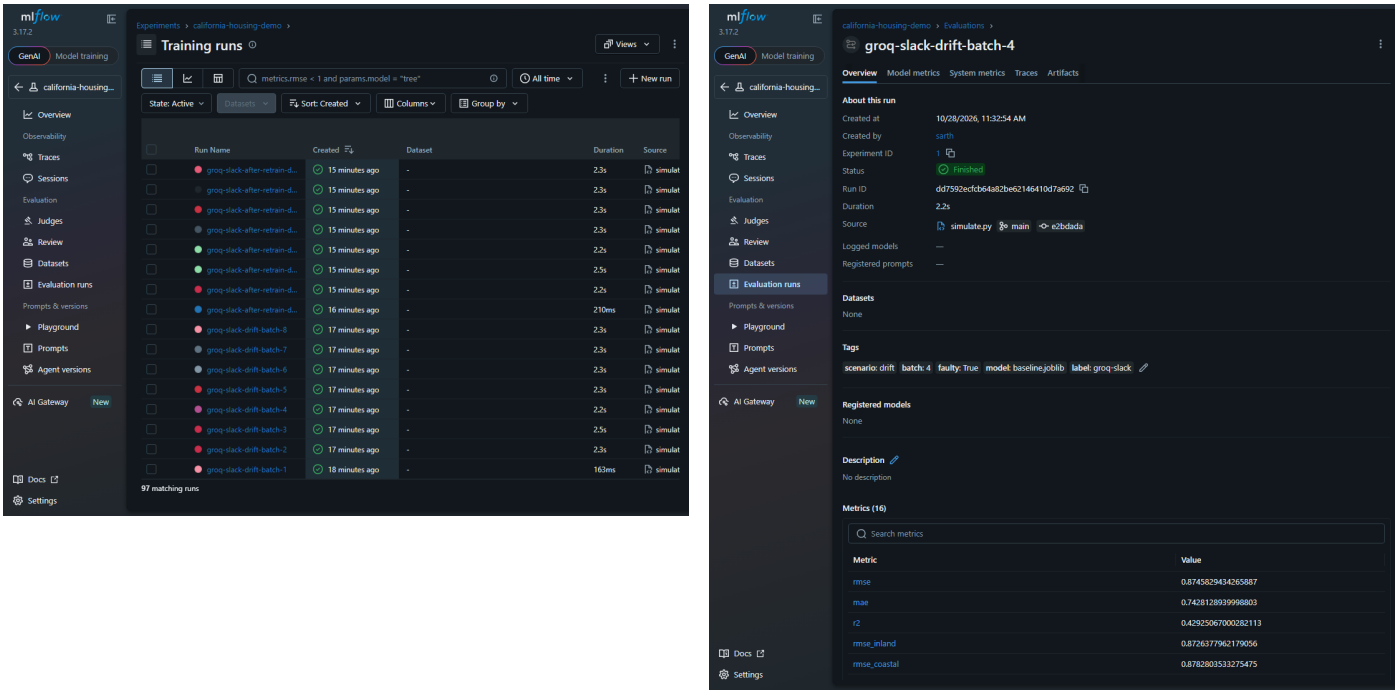


Fig. 5. Each fault's signal on the faulty batches for the original and the retrained model. Dashed lines are the limits.



(a) Runs logged by the simulator

(b) One faulty run: tags and metrics

Fig. 6. MLflow tracking server: (a) evaluation runs of the live Groq run and after retraining; (b) run groq-slack-drift-batch-4 with its tags (scenario drift, batch 4, faulty) and metrics (RMSE 0.875, R^2 0.429).

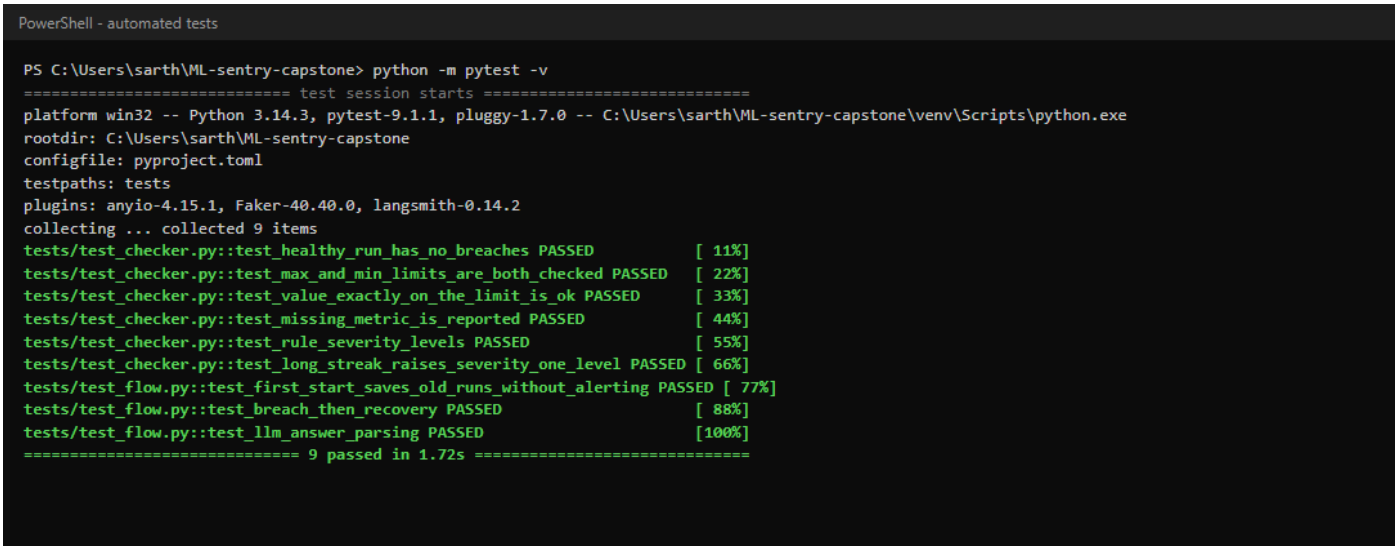


Fig. 8. The automated test suite (9 tests: threshold checks, severity rules, first-start history, breach-then-recovery, no double checking, LLM answer validation), all passing.